

5

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT
THÀNH PHỐ HỒ CHÍ MINH
KHOA CƠ KHÍ CHẾ TẠO MÁY
BỘ MÔN CƠ ĐIỆN TỬ

**ĐỀ THI CUỐI HỌC KỲ II NĂM HỌC
2024-2025**

Môn: Mạng thần kinh nhân tạo

Mã môn học: NNET331829E

Đề số/Mã đề: 01 Đề thi có 2 trang.

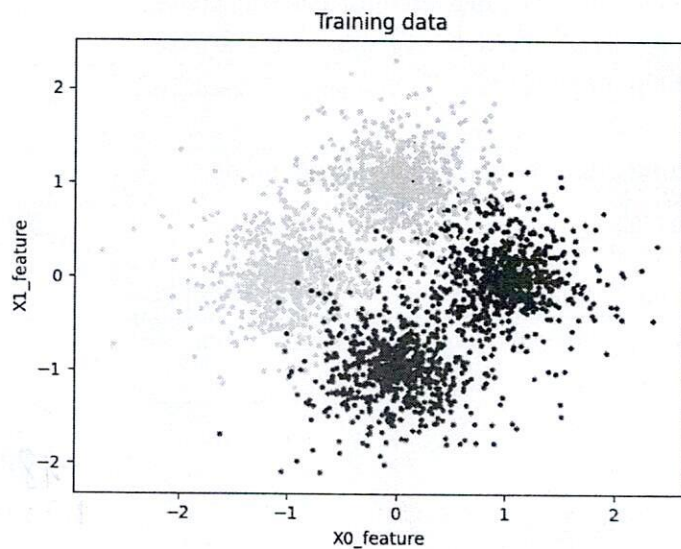
Thời gian: 90 phút.

Ngày thi: 19/05/2025

Được phép sử dụng tài liệu.

Câu 1: (2 điểm)

Viết chương trình python tạo lớp (class) PointZone cho phép khởi tạo dữ liệu điểm ngẫu nhiên có dạng đám mây điểm, với số lượng lớp (number of classes) và số lượng điểm trong mỗi lớp (number of points) được định nghĩa bởi người sử dụng. Ví dụ với 4 đám mây điểm (4 lớp), 800 điểm/lớp như hình dưới:



Câu 2: (2 điểm) Viết chương trình python định nghĩa class Dense cho phép khởi tạo các lớp Neurons. Trong class Dense, cũng như định nghĩa các hàm `__init__` (khởi tạo dense), `forward` (lan truyền thẳng) và `backward` (lan truyền ngược).

Câu 3: (2 điểm)

Viết chương trình python để định nghĩa các class sau, nhằm cho phép kích hoạt (**activate**) các lớp neuron, cũng như tính giá trị mất mát (**loss**) ở ngõ ra của ANN.

- **Class Loss:** Loss, Loss Categorical Cross Entropy.
- **Class Activation:** bao gồm Activation ReLU, Activation Softmax và Activation Softmax – Loss Categorical Cross Entropy.

Câu 4: (2 điểm)

Viết chương trình python cho phép định nghĩa *Adam optimizer* với chức năng learningrate_decay để tối ưu hóa các thông số của mạng neuron. Định nghĩa lần lượt các hàm `__init__` (khởi tạo), `pre_update_params` (trước cập nhật thông số), `update_params` (cập nhật thông số) và `post_update_params` (sau cập nhật thông số).

Câu 5: (2 điểm)

Viết chương trình python cho phép khởi tạo 1 mạng Neuron với 2 neurons ngõ vào, 2 lớp Dense (32 neurons / lớp), và 4 neurons ngõ ra cho phép huấn luyện để phân loại 4 lớp (classes) điểm khác nhau được tạo ra với class PointZone ở câu 1 (mỗi lớp có 800 điểm). Sau khi huấn luyện mạng neuron, tạo test data mới và hiển thị kết quả dự đoán với test data mới.

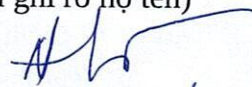
Ghi chú: Cán bộ coi thi không được giải thích đề thi.

Chuẩn đầu ra của học phần (về kiến thức)	Nội dung kiểm tra
[CLO2]: Trình bày được và lập trình được cấu trúc của nơ-ron, lớp nơ-ron & mạng thần kinh nhân tạo với nhiều lớp	Câu 2, 5
[CLO3]: Thiết lập được các Hàm kích hoạt (Activation) & Hàm mất mát (Loss)	Câu 3, 5
[CLO4]: Lập trình được chức năng lan truyền ngược (Back-Propagation)	Câu 3, 4, 5
[CLO5]: Thiết lập được các thuật toán tối ưu để cập nhật thông số mạng thần kinh nhân tạo	Câu 4, 5
[CLO7]: Vận dụng được các tập dữ liệu (kiểm tra, phê duyệt và huấn luyện mạng thần kinh nhân tạo)	Câu 1, 5

Ngày 18 tháng 05 năm 2025

1. Trưởng bộ môn

(ký và ghi rõ họ tên)


Võ Lâm Chương

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT
THÀNH PHỐ HỒ CHÍ MINH
KHOA CƠ KHÍ CHẾ TẠO MÁY
BỘ MÔN CƠ ĐIỆN TỬ

**ĐÁP ÁN - ĐỀ THI CUỐI HỌC KỲ II NĂM
HỌC 2024-2025**

Môn: Mạng thần kinh nhân tạo

Mã môn học: NNET331829E

Đề số/Mã đề: 01

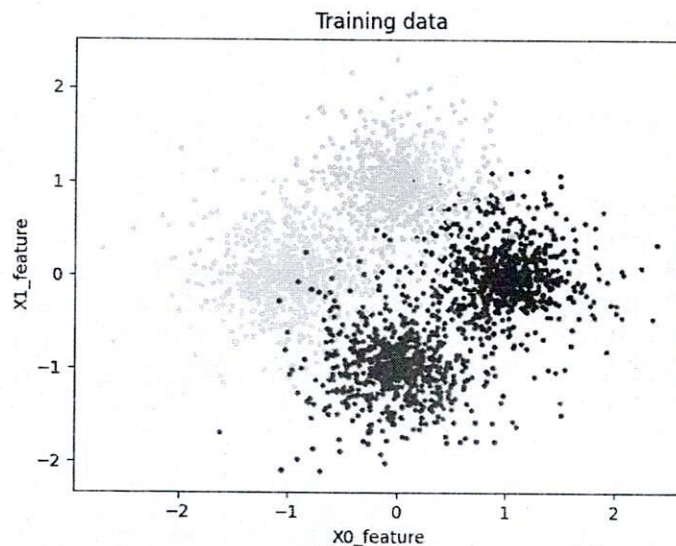
Thời gian: 90 phút.

Ngày thi: 19/05/2025

Được phép sử dụng tài liệu.

Câu 1: (2 điểm)

Viết chương trình python tạo lớp (class) PointZone cho phép khởi tạo dữ liệu điểm ngẫu nhiên có dạng đám mây điểm, với số lượng lớp (number of classes) và số lượng điểm trong mỗi lớp (number of points) được định nghĩa bởi người sử dụng. Ví dụ với 4 đám mây điểm (4 lớp), 800 điểm/lớp như hình dưới:



File Points.py:

```
import numpy as np
import matplotlib.pyplot as plt

class Zone:
    def __init__(self, n_points, n_classes, n_dimensions):
        #generating a randomized data
        self.N = n_points # number of points per class
        self.D = n_dimensions # dimension
        self.K = n_classes # number of classes
        self.P = np.zeros((self.N*self.K,self.D))
        self.L = np.zeros(self.N*self.K, dtype='uint8')
        pi = 3.1415
        for j in range(self.K):
```



```

        theta = j*(2*pi)/self.K
        a = np.cos(theta)
        b = np.sin(theta)
        ix = range(self.N*j, self.N*(j+1))
        r = np.zeros(self.N)
        for i in range(len(r)):
            r[i] = np.random.randn(1)*0.5
            t = np.linspace(0, 2*pi, self.N) +
np.random.randn(self.N)*0.2
            self.P[ix] = np.c_[a + r*np.sin(t), b + r*np.cos(t)]
            self.L[ix] = j

```

Câu 2: (1.5 điểm)

Viết chương trình python định nghĩa class Dense cho phép khởi tạo các lớp Neurons. Trong class Dense, cũng như định nghĩa các hàm `__init__` (khởi tạo dense), `forward` (lan truyền thẳng) và `backward` (lan truyền ngược).

File Dense.py:

```

import numpy as np

class Dense:
    def __init__(self, n_inputs, n_neurons):
        #init weights and biases
        self.weights = 0.01*np.random.randn(n_inputs, n_neurons)
        self.biases = np.zeros((1, n_neurons))

    def forward(self, inputs):
        #calculate outputs
        self.inputs = inputs
        self.output = np.dot(inputs, self.weights) + self.biases

    # backward pass
    def backward(self, dvalues):
        # Gradients on parameters
        self.dweights = np.dot(self.inputs.T, dvalues)
        self.dbiases = np.sum(dvalues, axis = 0, keepdims = True)
        # Gradients on values
        self.dinputs = np.dot(dvalues, self.weights.T)

```

Câu 3: (3 điểm)

Viết chương trình python để định nghĩa các class sau, nhằm cho phép kích hoạt (*activate*) các lớp neuron, cũng như tính giá trị mất mát (*loss*) ở ngõ ra của ANN.

- **Class Loss:** Loss, Loss Categorical Cross Entropy.
- **Class Activation:** bao gồm Activation ReLU, Activation Softmax và Activation Softmax – Loss Categorical Cross Entropy.

File Loss.py:

```

import numpy as np
import math

class Loss:
    def calculate(self, output, y):
        sample_losses = self.forward(output, y)
        data_loss = np.mean(sample_losses)

```



```

        return data_loss

class Loss_CategoricalCrossentropy(Loss):

    # Forward Pass
    def forward(self, y_pred, y_true):
        samples = len(y_pred)

        # Clip both sides to not drag mean toward any values
        y_pred_clipped = np.clip(y_pred, 1e-7, 1-1e-7)

        # Probabilities for target values only if categorical labels
        if len(y_true.shape) == 1: # The array is 1D
            correct_confidence = y_pred_clipped[
                range(samples),
                y_true
            ]
        elif len(y_true.shape) == 2: # The array is 2D
            correct_confidence = np.sum(
                y_pred_clipped*y_true,
                axis = 1
            )

        # Losses calculation
        negative_log_likelihoods = -np.log(correct_confidence)
        return negative_log_likelihoods

    # Backward Pass
    def backward(self, dvalues, y_true):
        samples = len(dvalues)
        labels = len(dvalues[0])
        # if labels are sparse, turn them into one vector
        if len(y_true.shape) == 1:
            y_true = np.eye(labels)[y_true]
        # Calculate gradient
        self.dinputs = - y_true / dvalues
        # Normalize gradient
        self.dinputs = self.dinputs / samples

```

File Activation.py:

```

import numpy as np
import Loss as loss

class Activation_ReLU:
    #Forward Pass
    def forward(self, inputs):
        #Calculate output values from input
        self.inputs = inputs
        self.output = np.maximum(0, inputs)
    # Backward Pass
    def backward(self, dvalues):
        self.dinputs = dvalues.copy()
        # Put to zero gradient where input is negative
        self.dinputs[self.inputs <= 0] = 0

class Activation_SoftMax():

    #Forward pass:

```



```

def forward(self,inputs):
    #Get unnormalized probabilities
    exp_values = np.exp(inputs - np.max(inputs, axis =
1,keepdims=True))
    self.exp_values = exp_values

    #Normalized them for each sample
    probabilities = exp_values / np.sum(exp_values, axis = 1,
keepdims=True)
    self.output = probabilities

    # Backward pass:
    def backward(self,dvalues):
        self.dinputs = np.empty_like(dvalues)
        # Enumerate outputs and gradients
        for index, (single_output,single_dvalues) in
enumerate(zip(self.output, dvalues)):
            single_output = single_output.reshape(-1,1)
            jacobian_matrix = np.diagflat(single_output) -
np.dot(single_output,single_output.T)
            self.dinputs[index] =
np.dot(jacobian_matrix,single_dvalues)

class Activation_Softmax_Loss_CategoricalCrossEntropy():

    # Create activation and loss function object
    def __init__(self):
        self.activation = Activation_SoftMax()
        self.loss = loss.Loss_CategoricalCrossentropy()

    # Forward pass
    def forward(self, inputs, y_true):
        # Output layer's activation function
        self.activation.forward(inputs)
        self.output = self.activation.output
        return self.loss.calculate(self.output, y_true)

    # Backward pass
    def backward(self, dvalues, y_true):
        samples = len(dvalues)

        #if labels are one-hot encoded, turn them into discrete
values
        if len(y_true.shape) == 2:
            y_true = np.argmax(y_true, axis =1)

        self.dinputs = dvalues.copy()
        self.dinputs[range(samples), y_true] -= 1
        self.dinputs = self.dinputs / samples

```

Câu 4: (1.5 điểm)

Viết chương trình python cho phép định nghĩa **Adam optimizer** với chức năng learningrate_decay để tối ưu hóa các thông số của mạng neuron. Định nghĩa lần lượt các hàm `__init__` (khởi tạo), `pre_update_params` (trước cập nhật thông số), `update_params` (cập nhật thông số) và `post_update_params` (sau cập nhật thông số).

File Optimizer.py:

```
import numpy as np

class Optimizer_Adam:

    def __init__(self, learning_rate = 0.1, decay = 5e-3, epsilon =
1e-7, beta1 = 0.9, beta2 = 0.9):
        self.learning_rate = learning_rate
        self.current_learning_rate = learning_rate
        self.decay = decay
        self.step = 0
        self.epsilon = epsilon
        self.beta1 = beta1
        self.beta2 = beta2

    # pre update
    def pre_update_params(self):
        if self.decay:
            self.current_learning_rate = self.learning_rate * (1 / (1
+ self.decay * self.step))

    # Update parameters
    def update_params(self, layer):

        if not hasattr(layer, 'weights_cache'):
            layer.weights_momentum = np.zeros_like(layer.weights)
            layer.weights_cache = np.zeros_like(layer.weights)
            layer.biases_momentum = np.zeros_like(layer.biases)
            layer.biases_cache = np.zeros_like(layer.biases)

        layer.weights_momentum = self.beta1 * layer.weights_momentum
+ (1 - self.beta1) * layer.dweights
        layer.biases_momentum = self.beta1 * layer.biases_momentum +
(1 - self.beta1) * layer.dbiases
        weights_momentum_corrected = layer.weights_momentum / (1 -
self.beta1**(self.step + 1))
        biases_momentum_corrected = layer.biases_momentum / (1 -
self.beta1**(self.step + 1))
        layer.weights_cache = self.beta2 * layer.weights_cache + (1-
self.beta2) * layer.dweights**2
        layer.biases_cache = self.beta2 * layer.biases_cache + (1-
self.beta2) * layer.dbiases**2
        weights_cache_corrected = layer.weights_cache / (1 -
self.beta2**(self.step + 1))
        biases_cache_corrected = layer.biases_cache / (1 -
self.beta2**(self.step + 1))
        layer.weights += -self.current_learning_rate *
weights_momentum_corrected / (np.sqrt(weights_cache_corrected) +
self.epsilon)
        layer.biases += -self.current_learning_rate *
biases_momentum_corrected / (np.sqrt(biases_cache_corrected) +
self.epsilon)

    # post update
    def post_update_params(self):
        self.step += 1
```


Câu 5: (2 điểm)

Viết chương trình python cho phép khởi tạo 1 mạng Neuron với 2 neurons ngõ vào, 2 lớp Dense (32 neurons / lớp), và 4 neurons ngõ ra cho phép huấn luyện để phân loại 4 lớp (classes) điểm khác nhau được tạo ra với class PointZone ở câu 1 (mỗi lớp có 800 điểm). Sau khi huấn luyện mạng neuron, tạo test data mới và hiển thị kết quả dự đoán với test data mới.

```
# Example of Training in loop: running for 10000 times

import numpy as np
import Activation as activation # type: ignore
import Dense as layer # type: ignore
import Optimizer as optimizer # type: ignore
import Points as point # type: ignore
import matplotlib.pyplot as plt

n_points = 800
n_classes = 4
Training_Data = point.Zone(n_points,n_classes,2)

# Visualization of data (2D):
plt.figure(1)
plt.scatter(Training_Data.P[:, 0], Training_Data.P[:, 1],
c=Training_Data.L, s=10, cmap=plt.cm.Spectral)
plt.xlabel("Feature X0")
plt.ylabel("Feature X1")
plt.title("Training data")
plt.show()

# Create the NN with 2 inputs, 2 hidden layers (32 neurons) and 3
outputs:
Dense1 = layer.Dense(2,32)
activation1 = activation.Activation_ReLU()
Dense2 = layer.Dense(32,32)
activation2 = activation.Activation_ReLU()
Dense3 = layer.Dense(32,n_classes)
loss_activation =
activation.Activation_Softmax_Loss_CategoricalCrossEntropy()

# Create the optimizer (to updates the NN's weights and biases)

# Adam parameters
learning_rate = 0.02
decay = 1e-4
epsilon = 1e-7
beta1 = 0.5
beta2 = 0.5

optimizer = optimizer.Optimizer Adam(learning_rate, decay, epsilon,
```



```

beta1, beta2)

# Train in loop
for epoch in range(10001):
    # Perform a forward pass
    Dense1.forward(Training_Data.P)
    activation1.forward(Dense1.output)
    Dense2.forward(activation1.output)
    activation2.forward(Dense2.output)
    Dense3.forward(activation2.output)

    # Calculate loss & accuracy
    loss = loss_activation.forward(Dense3.output, Training_Data.L)

    predictions = np.argmax(loss_activation.output, axis = 1)
    if len(Training_Data.L.shape) == 2:
        Training_Data.L = np.argmax(Training_Data.L, axis=1)
    accuracy = np.mean(predictions == Training_Data.L)

    # print if epoch % 100 = 0
    if not epoch % 100:
        print(f'epoch: {epoch}, ' +
              f'acc: {accuracy:.3f}, ' +
              f'loss: {loss:.3f} ' +
              f'LR: {optimizer.current_learning_rate:.3f}')

    # Back propagation
    loss_activation.backward(loss_activation.output, Training_Data.L)
    Dense3.backward(loss_activation.dinputs)
    activation2.backward(Dense3.dinputs)
    Dense2.backward(activation2.dinputs)
    activation1.backward(Dense2.dinputs)
    Dense1.backward(activation1.dinputs)

    # Update weights and biases using optimizer
    optimizer.pre_update_params()

    optimizer.update_params(Dense1)
    optimizer.update_params(Dense2)
    optimizer.update_params(Dense3)

    optimizer.post_update_params()

# Final result of classification of data
print(predictions) # Contain the information of classification of
the NN
print(Training_Data.L) # The training result vector

# Visualization of data (2D):
plt.figure(2)
plt.scatter(Training_Data.P[:, 0], Training_Data.P[:, 1],
c=predictions, s=10, cmap=plt.cm.Spectral)

```

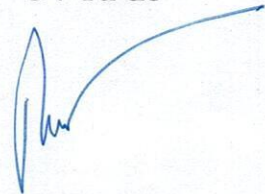


```
plt.title("Data predicted by the NN")
plt.xlabel("Feature X0")
plt.ylabel("Feature X1")
plt.show()
```

Ghi chú: Cán bộ coi thi không được giải thích đề thi.

Chuẩn đầu ra của học phần (về kiến thức)	Nội dung kiểm tra
[CLO2]: Trình bày được và lập trình được cấu trúc của nơ-ron, lớp nơ-ron & mạng thần kinh nhân tạo với nhiều lớp	Câu 2, 5
[CLO3]: Thiết lập được các Hàm kích hoạt (Activation) & Hàm mất mát (Loss)	Câu 3, 5
[CLO4]: Lập trình được chức năng lan truyền ngược (Back-Propagation)	Câu 3, 4, 5
[CLO5]: Thiết lập được các thuật toán tối ưu để cập nhật thông số mạng thần kinh nhân tạo	Câu 4, 5
[CLO7]: Vận dụng được các tập dữ liệu (kiểm tra, phê duyệt và huấn luyện mạng thần kinh nhân tạo)	Câu 1, 5

Ngày 18 tháng 5 năm 2025
GV ra đề



Cái Việt Anh Dũng